

· KLDP.org · KLDP Wiki · KLDP.net · [통합 검색](#) ·



[Subversion Book/Basic Concepts](#)



[FrontPage](#) | [FindPage](#) | [TitleIndex](#) | [RecentChanges](#) |

[SubversionBook](#) › [SubversionBookRemake](#) ›
[SubversionBook/Preface](#) › [SubversionBook/Introduction](#) ›
[SubversionBook/BasicConcepts](#)

Login:

Password:

[Join](#)

The best prophet of the future is the past.

[ThemeCoding](#)
[xmalloc](#)
[WindowsCE](#)
[SubversionBook/Preface](#)

1Chapter. 기본 개념

Table of Contents

- 1.1.
- 1.1. [저장소\(repository\)](#)
- 1.2. [버전 관리 모델](#)
 - 1.2.1. [파일 공유의 문제](#)
 - 1.2.2. [락-수정-언락 방법](#)
 - 1.2.3. [복사-수정-머지 방법](#)
- 1.3. [실행중의 Subversion](#)
 - 1.3.1. [작업 카피본](#)
 - 1.3.2. [리비전](#)
 - 1.3.3. [작업 카피본은 어떻게 저장소\(repository\)를 뒤쫓을까.](#)
 - 1.3.4. [혼합 리비전의 제약](#)
- 1.4. [개요](#)



kss@kldp.org

1.1.

이 장에서는 Subversion의 기본개념을 간단하게 설명합니다. 버전 컨트롤 시스템을 처음 사용하는 사람은 이 장을 읽어 주십시오. 일반적인 버전 관리의 개념을 시작으로 Subversion에만 해당되는 세부적인 개념을 설명하고, Subversion을 사용하는 간단한 예를 보여드릴 것입니다.

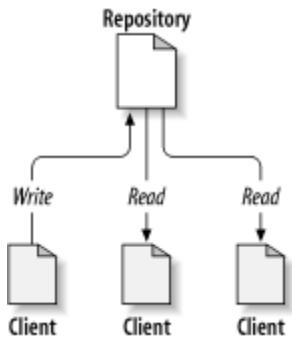
이 장에서는 프로그램 소스 코드들을 공유하는 예를 들어 설명하지만, Subversion은 프로그램 소스뿐만 아니라 어떤 종류의 파일들도 관리할 수 있습니다. Subversion은 컴퓨터 프로그래머만 쓰기 위해 만든 도구는 아닙니다.

1.1. 저장소(repository)

*Subversion*은 정보를 한 곳에 모아 집중적으로 관리하는 시스템입니다. 여기서 가장 중요한 핵심은 저장소(repository)라고 불리우는 데이터의 창고입니다. 저장소(repository)는 정보를 *파일 시스템 트리*, 즉, 일반적인 파일과 디렉토리 계층 구조의 형태로 보관합니다. 여러 클라이언트가 저장소(repository)에 접속 하여 파일을 읽고 수정하여 쓸 수 있습니다. 한 *클라이언트*가 데이터를 저장소에 쓰는 것은 정보를 다른 클라이언트들이 이용할 수 있도록 내놓은 것이고, 저장소로부터 데이터

를 읽는 것은 반대로 다른 클라이언트들이 내놓은 정보를 받는 것입니다. 아래의 그림 2.1은 이러한 전형적인 클라이언트/서버 시스템을 나타낸 것입니다.

Figure 1-1. 전형적인 클라이언트/서버 시스템



그럼 이런 일에 무슨 색다른 것이 있습니까? 여기까지 전형적인 파일 서버의 정의에 지나지 않는다고 생각합니다. 그리고 실제 저장소(repository)는 파일 서버의 **일종**입니다. 하지만 보통 말하는 파일서버와는 조금 다릅니다. Subversion의 저장소(repository)는 **수정내용을 모조리 기억**하고 있습니다. 각각의 파일에 대한 변경사항은 물론이고, 파일이나 디렉토리의 추가, 삭제, 재배치등의 디렉토리 트리의 변경도 모두 기억됩니다.

클라이언트가 저장소(repository)로부터 데이터를 읽어들이는 때는 보통은 파일 시스템 트리의 마지막 버전만이 보입니다. 하지만 파일 시스템의 **이전상태**도 열람할 수 있습니다. 예를 들어 클라이언트는 지난 주의 수요일에 이 디렉토리에는 어느 파일이 있었지? 라든지 마지막에 이 파일 변경한 것은 누구고 그 사람은 무엇을 변경했지? 등의 이력에 관한 질문을 할 수가 있습니다. 이 질문은 모든 **버전 관리 시스템**에서 가장 중요한 질문들입니다. 즉 버전 컨트롤 시스템이란 시간에 따라 데이터의 변화를 기록하여 수정내역을 추적할 수 있도록 만들어진 시스템입니다.

1.2. 버전 관리 모델

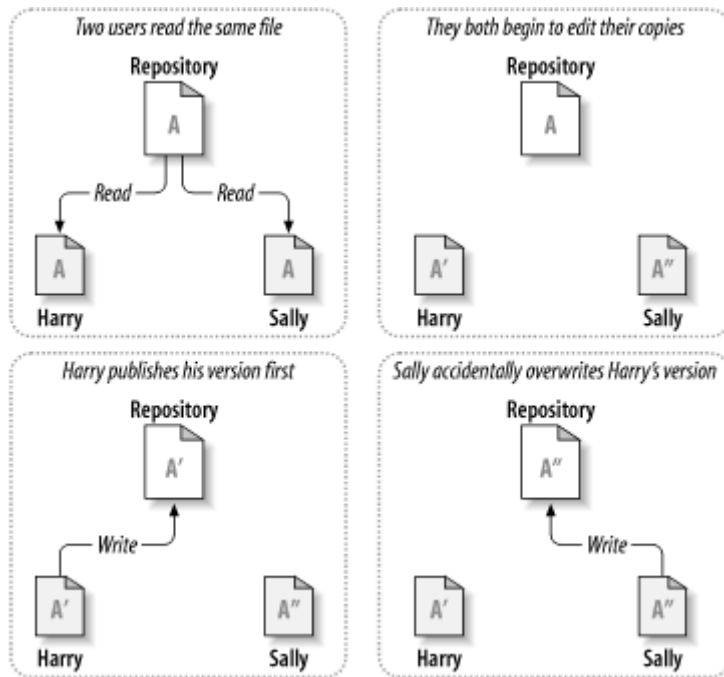
1.2.1. 파일 공유의 문제

버전 컨트롤 시스템이라면 어떤 것이라도 기본적인 문제 하나를 풀지 않으면 안됩니다: 어떻게 사용자들이 정보를 공유하면서도, 서로 수정을 겹치지 않게 하는가 입니다. 저장소(repository)에 다른 사람이 변경해 넣은 것을 잘못하여 덮쳐버리는 일은 간단히 일어날 수 있습니다.

이런 상황을 생각해 보십시오: 두 명의 동료 Harry와 Sally가 있습니다. 두 사람은 동시에 같은 저장소(repository)내의 파일을 편집하기로 했습니다. 만약 Harry가 먼저 그가 변경한 것을 저장소(repository)에 입력하고 나중에 Sally는 잘못하여 Harry의 것을 새로운 버전으로 덮쓰기해 버립니다. Harry의 버전은 영구히 없어지지 않습니다. (라고 말하는 것은 버전 컨트롤 시스템은 모든 변경을 기록하고 있기 때문에) Harry가 한 수정은 모두 Sally의 새로운 버전에는 **나타나는 일**이 없습니다. 편집시에는 그는 Harry의 변경을 볼 수 없기 때문입니다. Harry의 작업은 실질적으로 없어져 버립니다. 혹은 적어도 최신의 버전으로부터는 없어져 버립니다. 게다가 그것은 두 명이 의도한 것은 아닙니다.

이것이야말로 우리가 피하지 않으면 안 되는 상황입니다.

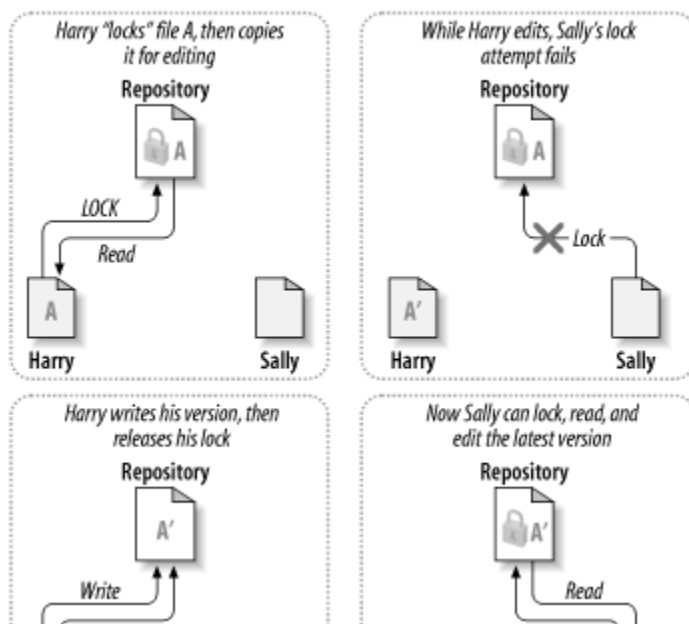
Figure 1-2. 피하지 않으면 안 되는 문제



1.2.2. 락-수정-언락 방법

많은 버전 관리 시스템에서는 락-수정-언락의 모델을 사용해 이 문제를 취급합니다만 이것은 매우 단순한 해결 방법입니다. 그러한 시스템에서는 저장소(repository)중의 파일을 변경할 수 있는 것은 한 번에 한 명 뿐입니다. 최초 Harry는 파일 변경하기 전에 락 하지 않으면 안됩니다. 파일의 락은 도서관으로부터 책을 빌리는 것과 비슷합니다. 만약 Harry가 파일을 잠그면 Sally는 같은 파일을 변경할 수 없게 됩니다. 잠그려고 하면 저장소(repository)는 그 요구를 거부합니다. Sally가 할 수 있는 것은 그 파일을 읽는 것으로 Harry가 일을 끝내 락 해제해 주는 것을 기다리는 것 만 입니다. Harry 가 락을 해제한 뒤 Harry의 차례는 끝나면 이번에 Sally가 잠그어 편집할 수가 있는 차례가 됩니다.

Figure 1-3. 락-수정-언락 방법





락-수정-언락의 모델의 문제는 파일 관리가 조금 너무 어려워 자주 사용자에게 작업의 방해가 됩니다:

- **잠그는 것은 관리상의 문제를 일으킬지도 모릅니다.** 가끔 Harry는 파일을 잠근 후에 그것을 잊어 버립니다. 한편 Sally 는 쭉 자신의 차례를 기다리고 있으므로 그 사이 굳이 하는 것이 할 수 없습니다. 그리고 Harry는 그대로 바캉스에 가 버려 Sally로서는 관리자에 대해서 Harry가 건 락을 해제하도록 부탁하지 않으면 안됩니다. 이 상황은 불필요한 지연과 시간낭비를 일으킵니다.
- **락은 불필요한 직렬화를 일으킬지도 모릅니다.** Harry는 그 텍스트 파일의 선두의 부분을 수정하고 Sally는 같은 파일의 마지막 부분을 수정하고 싶은 것뿐이라고 하면 두 사람의 수정은 완전히 겹치지 않습니다. 적당한 형태로 머지 되는 것 조차 보증할 수 있으면 두 명은 같은 파일을 동시에 편집할 수가 있어 그것이 큰 문제는 되지 않을 것입니다.
- **락은 잘못된 의미의 안심감을 주어 버리는 경우가 있습니다.** Harry가 파일 A를 잠그고 나서 편집하고 Sally는 동시에 파일 B를 잠그고 나서 편집하고 있다고 합니다. 그러나 여기서 A와 B와는 의미적으로 서로 의존하고 있어 각각 대하는 독립한 변경은 양립하지 않는다고 합시다. 돌연 A와 B는 이제 함께 동작하지 않게 됩니다. 락을 사용한 시스템은 이러한 상황에는 무력합니다. 이것은 어떤 의미로 잘못된 의미의 안심감을 주고 있습니다. Harry나 Sally가 파일을 잠그는 일로 각각 안전한 상태에 들어가 자신의 작업은 타인으로부터 분리되어 있다고 착각하는 것은 간단히 일어날 수 있습니다. 이것이 최초로 말한 것 같은 양립하지 않는 변경에 대한 논의를 방해해 버릴지도 모릅니다.

1.2.3. 복사-수정-머지 방법

Subversion, CVS와 몇몇 다른 버전 컨트롤 시스템은 락킹 대신에 복사-수정-머지모델을 사용합니다. 이 모델에서는 사용자마다의 클라이언트 머신은 저장소(repository)를 읽어들여 파일이나 프로젝트에 대한 자신만의 작업 카피본을 만듭니다. 그리고 사용자는 각각이 자신의 작업 카피본을 수정합니다. 그리고 마지막에 각각의 카피본은 최종 버전에 머지됩니다. 이러한 버전 관리 시스템 대부분은 머징작업에 도움을 주지만 최종적으로는 올바른 머지인지 아닌지는 사람이 책임을 져야 합니다.

예를 들어보겠습니다. Harry와 Sally가 저장소(repository)의 내용을 복사해 한 프로젝트에 대해 각각의 작업 카피본을 만들었다고 합시다. 둘은 동시에 각자의 작업 카피본중 파일 A를 수정합니다. Harry가 Sally 보다 늦게 자신의 수정분을 보존하려고 할 때 저장소(repository)는 Harry가 가진 파일 A는 *최신이 아니다*라고 알려줍니다. 바꾸어 말하면 저장소(repository)에 현재 있는 파일 A는 그가 그것을 복사해 간 후에 다른 사람에 의해 수정되었다는 말입니다. Harry는 Subversion 클라이언트 프로그램에게 자신의 작업 카피본 A과 저장소에 있는 현재 버전의 차이점을 *머지* 하도록 명령합니다. Sally가 수정한 부분과 Harry가 수정한 부분이 겹치지 않는다고만 하면, 한번에 Harry가 두가

지 수정분을 적용해서 머징하기만 하면 작업 카피본을 저장소(repository)에 다시 써 넣을 수 있습니다. 아래 그림 2.4 복사-수정-머징 해법과 그림 2.5 복사-수정-머징 해법 (계속) 가 여기에서 설명한 절차를 나타내고 있습니다.

Figure 1-4. 복사-수정-머지 방법

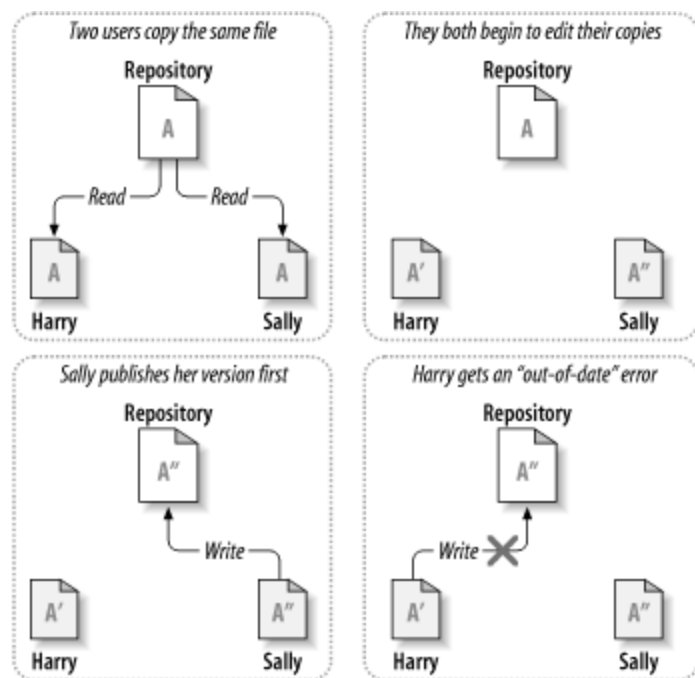
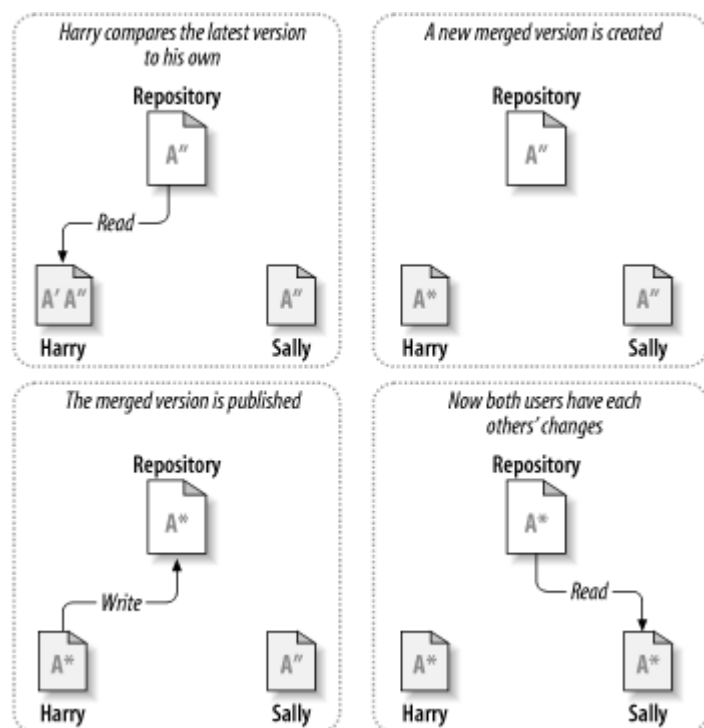


Figure 1-5. 복사-수정-머지. 계속.



그러나 Sally가 수정한 부분이 Harry가 수정한 부분과 겹쳐 있다면 어떻게 될까요? 이런 상황을 충돌이라고 부르며 일반적인 경우 큰 문제는 되지 않습니다. Harry가 Subversion 클라이언트 프로그램에 저장소(repository)의 최신의 변경을 자신의 작업 카피본에 머지 하도록 요구했을 때 그의 A파일의 작업 카피본은 충돌 상태로서 마크 됩니다. 그는 양쪽 모두의 변경의 충돌한 부분을 볼 수가 있어 어느 쪽을 선택할지 선택합니다. 소프트웨어 자체는 자동적으로 충돌을 해결해 주지 않는 것

이 주의해 주십시오; 인간만이 문제를 이해하고 올바르게 선택하는 힘을 가지고 있습니다. Harry가 일단 겹치는 부분의 수정을 직접 해결한 다음에 (예를 들어 Sally와 충돌에 대해 이야기한 후에) 머지 된 파일을 저장소(repository)에 안전하게 다시 써 넣어 저장할 수 있습니다.

복사-수정-머지 모델은 조금 혼란스럽게 들릴지도 모르겠지만 실제로 작업은 매우 매끄럽게 이루어질 수 있습니다. 사용자는 동시에 작업을 할 수 있고 상대의 수정을 기다릴 필요는 없습니다. 같은 파일을 변경할 때도 대부분의 변경은 전혀 겹치지 않는다는 것을 압니다. 그리고 충돌을 해소하는데 걸리는 시간은 시스템을 잠궜서 낭비되는 시간보다 훨씬 짧습니다.

결국 모든 문제는 한가지로 귀결됩니다. : 사용자 사이의 커뮤니케이션입니다. 사용자가 그다지 서로 의견의 교환을 하지 않으면 양쪽 모두의 구문 또는 의미상 충돌은 증가합니다. 어떤 시스템도 사용자에게 완벽한 커뮤니케이션을 강요할 수 없고, 의미상의 충돌을 검출해 줄 수도 없습니다. 그러한 이유로 잠그는 시스템(locking system, Sourcesafe 같은 도구)이 충돌을 피할 수가 있다고 하는 잘못된 보증에 안심할 이유는 없습니다. 실제로는 락은 생산성을 떨어뜨리는 것 이외에는 아무것도 아닙니다.

1.3. 실행중의 Subversion

1.3.1. 작업 카피본

이미 독자들이 작업 카피본에 대해 읽었으리라 보고 Subversion 클라이언트가 작업 카피본을 만들고 사용하는 모습을 보겠습니다.

Subversion 작업 카피본은 자신의 로컬 시스템상의 보통 디렉토리 트리로 되어 있고, 그 안에는 파일들이 들어 있습니다. 당신이 편집할 수 있는 파일이 소스 코드 파일이라면 그것은 컴파일 할 수 있습니다. 작업 카피본은 자신만의 작업 영역입니다: Subversion에게 그러라고 하기 전까지 자신의 작업 카피본은 다른 사람의 변경의 영향을 받지 않고, 또한 자신의 수정이 남에게 보여지지도 않습니다.

작업 카피본의 파일을 수정하고 잘 동작하는 것을 확인한 후에 Subversion은 그 변경된 점을 같은 프로젝트의 당신과 함께 작업하고 있는 다른 사람에게 공개하기 위한 커멘트를(저장소(repository)에 커밋하는 것) 준비합니다. 만약 다른 사람이 자기 자신의 변경을 공개했을 때 에는 Subversion은 그 변경을 자신의 작업 카피본에 머지하는 커멘드를 준비합니다. (저장소(repository)의 내용을 읽어들이는 것)

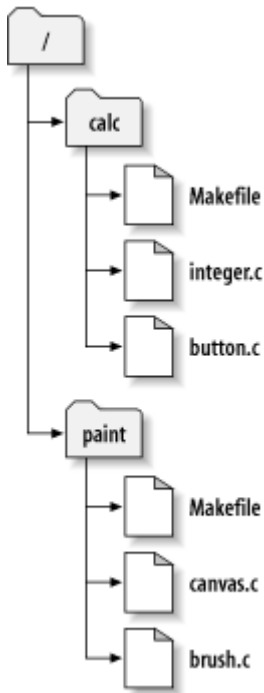
작업 카피본에는 Subversion에 의해 관리되는 몇개의 특수한 파일이 있습니다. 그 파일들에 의해(읽고 쓰기 등의) 커멘트를 실행합니다. 특히 작업 카피본 디렉토리에는 .svn이라는 이름의 관리 디렉토리가 서브 디렉토리로 있습니다. *관리 디렉토리*의 각각의 파일은 Subversion이 어느 파일에 아직 공개하고 있지 않는 변경된 점이 있는지 어느 파일이 다른 사람의 작업에 의해 최신인지 아닌지를 이해하는 것을 돕는 것입니다.

전형적인 Subversion 저장소(repository)는 복수의 프로젝트의 파일 (또는 소스 코드)를 저장하고 있습니다. 보통 각각의 프로젝트는 저장소(repository)의 파일 시스템 트리중의 서브 디렉토리로 되어 있습니다.

이 구성에 의해 사용자의 작업 카피본은 보통 저장소(repository)의 특정 부분에 대응하고 있습니다.

예를 들어 두 개의 소프트웨어 프로젝트를 포함한 저장소(repository)가 있다고 합시다.

Figure 1-6. 저장소(repository)의 파일 시스템



말을 바꾸어 말하면 저장소(repository)의 최상위의 디렉토리에는 두 개의 서브 디렉토리 : paint와 calc 가 있습니다.

작업 카피본을 가지고 오기 위해 저장소(repository)중의 서브 트리를 체크아웃하지 않으면 안됩니다. (check out 이라는 말은 무엇인가를 잠그거나 보호하는 것 같은 의미가 있습니다만 그런것 같지는 않습니다; 그것은 단지 자신을 위한 프로젝트의 복사를 만들 뿐입니다.) 예를 들어 /calc를 체크아웃 하는 것으로 작업 카피본을 가져올 수 있습니다.:

```
$ svn checkout http://svn.example.com/repos/calc
```

```
A calc
A calc/Makefile
A calc/integer.c
A calc/button.c
```

```
$ ls -a calc
Makefile integer.c button.c .svn/
```

A의 문자로 시작되는 일람은 Subversion이 당신의 작업 카피본에 몇 개의 파일을 추가했는지 나타내고 있습니다. 이것으로 저장소(repository)에 있는 /calc디렉토리의 작업 카피본을 가지고 올 수 있었습니다. 최초로 말한 것처럼 체크아웃시에는 또 하나의 .svn 디렉토리가 생깁니다. 이것은 Subversion에 필요한 추가 정보를 저장하는 장소가 됩니다.

저장소(repository)의 URL

Subversion의 저장소(repository)에는 여러가지 방법으로 접근하는 것이 가능합니다. 저장소는 로컬 디스크에 있을수도 있고 다양한 네트워크 프로토콜을 통해서 접근할 수도 있습니다. 그러나 어떠한 경우에서도 저장소(repository)의 장소는 항상 URL에 의해 표현됩니다. URL에는 액세스 방법을 나타내는 부분이 있습니다:

Table 1-1. 저장소(repository)에 액세스하기 위한 URL

대부분의 경우 Subversion의 URL는 표준적인 구문을 사용해서 서버 명칭과 포트 번호를 지정할 수 있습니다. file:의 액세스 방법은 서버가 클라이언트와 같은 머신상에 있는 경우만 유효합니다. — 실제 규약에 따라 URL의 서버 명칭의 부분은 존재하지 않는가 localhost여야 합니다:

```
$ svn checkout file:///path/to/repos
```

```
$ svn checkout file://localhost/path/to/repos
```

또한 Windows 플랫폼에서 같은 머신에 있지만 현재 작업 공간과 저장소가 서로 다른 드라이브에 있는 상태에서 file:의 구문을 사용하기 위해서는 비공식의 "표준적인" 구문을 사용해야 합니다. 이하의 두 개의 URL 패스 구문을 사용할 수 있습니다. 여기서 X는 저장소(repository)가 있는 드라이브입니다:

```
C:\W> svn checkout file:///X:/path/to/repos
```

```
C:\W> svn checkout "file:///X|/path/to/repos"
```

두번째의 구문에서는 세로 선이 pipe 의 의미로 해석되는 것을 방지하기 위해 URL 을 따옴표로 인용해야 합니다.

Windows 상의(URL이 아닌) 경로명에는 보통 backslash를 사용하지만 여기서는 URL을 일반적인 slash를 사용해 주세요.

button.c로 변경을 더하는 것을 생각해 보겠습니다. .svn 디렉토리는 파일의 수정 시각과 원래의 내용을 기억하고 있으므로 Subversion은 그 파일에 어떤 변경이 되었는지를 표시 할 수 있습니다. 그러나 Subversion은 명시적으로 표시해 줘라고 말해줄 때까지 그 변경을 표시 하지 않습니다. 자신의 변경을 공개하는 조작을 변경점의 커밋(혹은 체크인)이라고 말합니다.

변경점을 다른 사람에게 공개하려면 Subversion의 **commit** 커멘드를 사용합니다:

```
$ svn commit button.c
Sending button.c
```



```
Transmitting file data..
Committed revision 57.
```

이것으로 button.c에의 변경은 저장소(repository)에 커밋되었습니다. 만약 다른 사용자가 /calc의 작업 카피본을 만드는데 체크아웃 하면 최신 버전중에 당신의 변경점을 보게 되겠지요.

함께 작업하고 있는 Sally가 당신이 체크아웃 한 것과 같은 시각에 /calc의 작업 카피본을 자신용으로 체크아웃 했다고 합시다. 당신이 button.c에의 자신의 변경을 커밋해도 Sally의 작업 카피본은 변경되지 않는 상태인 채입니다. Subversion은 사용자의 요구에 따라서는 글자재라고 작업 카피본의 내용을 변경합니다.

작업 내용을 프로젝트의 최신 상태로 하려면 Sally는 Subversion에 자신의 작업 카피본을 갱신 하도록(듯이) 의뢰하지 않으면 되지 않습니다. 이것에는 **update** 커멘트를 사용합니다. 이것은 당신의 변경을 그녀의 작업 카피본에 머지 하고 그녀가 체크 아웃 한 후에 다른 사람이 커밋한 모든 부분에 대해서도 머지 합니다.

```
$ pwd
/home/sally/calc

$ ls -a
.svn/ Makefile integer.c button.c

$ svn update
U button.c
```

svn update 커멘드로부터의 출력은 Subversion이 button.c의 내용을 갱신한 것을 나타내고 있습니다. Sally는 어느 파일을 갱신 하는지를 지정할 필요가 없는데 주의해 주세요. Subversion은 .svn디렉토리의 정보와 저장소(repository)의 정보를 사용해 어느 파일을 갱신하지 않으면 안 되는가 (을)를 결정합니다.

1.3.2. 리비전

svn commit조작은 하나의 트랜잭션(transaction)으로서 임의의 수의 파일, 디렉토리에 대한 변경점을 공개하는 일이 생깁니다. 작업 카피본 중에서 파일의 내용을 바꾸거나 새롭다 파일을 만들거나 삭제하거나 이름을 바꾸거나 파일이나 디렉토리를 복사한 뒤 그러한 변경점의 전체를 완전한 사람 덩어리의 것으로 해 커밋할 수가 있습니다.

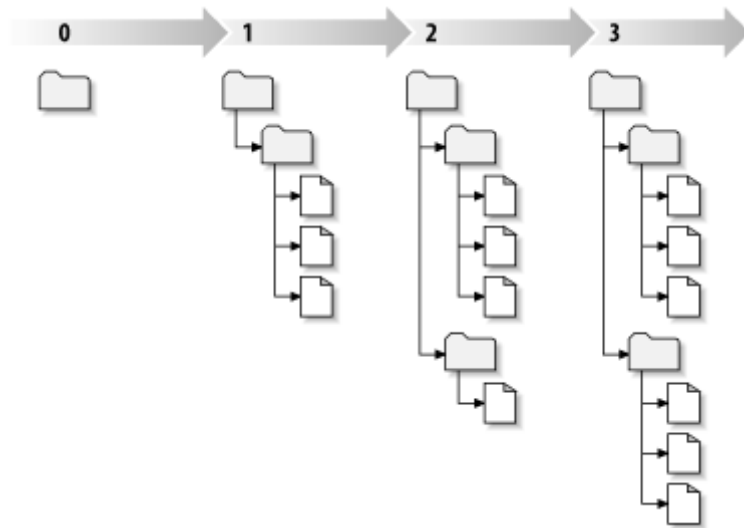
저장소(repository)에서는 각각의 커밋은 하나의 분할할 수 없는 사람 덩어리의 트랜잭션(transaction)로서 취급합니다. 모든 커밋에 변경은 완전하게 실행되는지 전혀 실행되지 않는가의 어느 쪽인지입니다. Subversion은 이 불분할의 성질을 프로그램 장애, 시스템 장애, 네트워크 장애등 그 외의 조작이 있는 경우에도 유지하려고 합니다.

저장소(repository)가 커밋을 받아들일 때는 항상 *리비전*으로 불리는 파일 시스템 트리의 새로운 상태를 만듭니다. 각각의 리비전에는 일의 인 자연수를 할당할 수 있습니다. 전의 버전보다 후의 버전이 수가 커집니다. 저장소(repository) 신규 작성시의 최초의 버전은 제로로 루트 디

렉토리 이외로는 아무것도 포함되어 있지 않습니다.

저장소(repository)를 시각화하는 능숙한 방법은 일렬에 줄선 나무로서 파악한다 일입니다. 0으로부터 시작하는 리비전 번호가 왼쪽에서 오른쪽으로 추가되어 간다 상황을 상상해 주세요. 각각의 리비전 번호에는 대응했다 파일 시스템목이 있어 각각의 나무는 각각의 커밋 후의 저장소(repository) 상태를 나타내는 "snapshot" 입니다.

Figure 1-7. 저장소(repository)



글로벌 리비전 번호

다른 많은 버전 컨트롤 시스템과는 달리 Subversion의 리비전 번호는 **전체 트리**에 대해서 붙여지는 것으로 개개의 파일에 붙여지는 것이 아닙니다. 각각의 리비전 번호는 전체 트리에 대해 몇번의 커밋에 의한 변경 후의 저장소(repository)의 특정 상태를 나타내는 것입니다. 리비전 N은 N번째의 커밋 후의 저장소(repository) 파일 시스템 상태를 나타는 것이라고 생각하면 됩니다. Subversion 사용자가 ``foo.c의 리비전 5"라고 말할 때 그것이 실제로 의미하는 것은 ``리비전 5에 있는 foo.c"입니다. 일반적으로 어느 파일의 리비전 N과 M이 꼭 차이가 있어야 할 필요는 **없습니다**! CVS는 파일마다 리비전 번호를 사용하므로 이 차이를 자세하게 알고 싶은 사람은 Appendix A CVS 사용자를 위한 SVN를 봐 주세요.

작업 카피본은 항상 저장소(repository)의 어떤 것이나 하나의 리비전 대응하고 있다고는 한정되지 않는 것에 주의해 주세요. 작업 카피본은 여러개의 다른 리비전으로부터 얻은 파일 (을)를 포함하고 있을지도 모릅니다. 예를 들어 최신 리비전 번호가 4인 저장소(repository)로부터 작업 카피본을 체크아웃 했다고 합시다:

```
calc/Makefile:4
integer.c:4
button.c:4
```

이 시점에서는 작업 카피본은 저장소(repository)의 리비전 4와 일치하고 있습니다. 그러나 여기서 button.c로 변경을 더하고 그 변경을 커밋

했다고 합니다. 그 밖에 커밋한 사람이 없는 경우 이번 커밋은 저장소(repository)의 버전을 5에 주어 작업 카피본의 내용은 아래와 같이 됩니다.

```
calc/Makefile:4
    integer.c:4
    button.c:5
```

이 시점에서 Sally가 integer.c에 대한 수정을 커밋해 리비전 6을 만들었다고 합시다. 이때 **svn update** 커맨드로 당신의 작업 카피본을 갱신하면 다음과 같이 되겠지요:

```
calc/Makefile:6
    integer.c:6
    button.c:6
```

Sally의 integer.c에의 변경은 당신의 작업 카피본에 나타나지만 button.c에 대한 당신의 변경은 그대로입니다. 이 예에서는 Makefile의 텍스트는 리비전 4,5,6으로 완전히 동일한 것이지만 Subversion은 당신의 작업 카피본중의 Makefile의 리비전을 6으로서 그것이 최신인 것을 표현합니다. 그래서 자신의 작업 카피본을 완전히 업데이트를 했을 때에는 일반적으로 작업 카피본은 저장소(repository)에 있는 특정 리비전과 완전하게 일치합니다.

1.3.3. 작업 카피본은 어떻게 저장소(repository)를 뒤쫓을까.

작업 카피본중의 각각의 파일에 대해 Subversion은 두 본질적인 정보를. .svn/관리 영역에 기록합니다:

- 당신의 작업 파일은 어느 리비전에 근거하고 있을까 (이것은 파일의 *작업 리비전*으로 불립니다) 그리고
- 저장소(repository)에 의해 작업 카피본가 마지막에 갱신되었을 때 각

저장소(repository)와의 교환으로 얻은 이 정보로 Subversion은 작업 파일의 각각이 이하의 4개 상태의 어느 것으로 있을까를 말할 수가 있습니다:

변경되지 않음 최신

작업 디렉토리의 파일은 변경되어 있지 않고 그 작업 리비전 이후에 일어난 저장소(repository)에 대한 커밋에서도 그 파일에 대한 변경이 없는 상태. 그 파일에 대한 **svn commit**을 실행해도 아무것도 하지 않고 **svn update** 명령도 아무것도 하지 않습니다.

최신 변경점

작업 카피본에 있는 파일이 변경되었고 그 파일의 베이스 리비전 이후에 저장소(repository)에 있는 파일은 변경되지 않은 상태. 작업 카피본에 아직 커밋하고 있지 않는 변경이 있으므로 그 파일

에 대한 **svn commit**는 당신의 변경점을 그대로 저장소에 반영하고 성공할 것입니다. **svn update**는 아무것도 하지 않습니다.

변경되지 않음 최신은 아님

작업 디렉토리에 있는 파일이 변경되지는 않았으나 저장소(repository)에는 변경이 있었습니다. 이 파일은 최신 리비전과 동일하게 만들기 위해서 갱신할 필요가 있습니다. 그 파일에 대한 **svn commit** 커멘트는 아무것도 하지 않습니다. 그 파일에 대한 **svn update**는 당신의 작업 복사에 최신의 수정점을 적용합니다.

변경됨 최신 아님.

파일은 작업 카피본에서도 저장소(repository)에서도 변경되었습니다. 파일에 대한 **svn commit**는 out-of-date 에러를 일으킵니다. 먼저 그 파일을 갱신해야 합니다. 파일에 대한 **svn update**는 공개된 변경점을 작업 카피본의 변경과 합치려고 할 것입니다. Subversion이 자동으로 합칠 수 없을 경우 사용자가 충돌을 해결하도록 그대로 둡니다.

이것에는 여러가지 정보의 변화를 쫓을 필요가 있다고 생각합니다만 **svn status** 커멘드를 사용하면 당신의 작업 카피본에 있는 모든 파일의 상태를 표시할 수 있습니다. 이 커멘트에 대한 보다 자세한 정보는 **svn status > (을)**를 봐 주세요.

1.3.4. 혼합 리비전의 제약

Subversion은 할수있는 한 유연하게 동작하는 것은 원칙으로 삼고 있습니다. 이 특별한 예로서 작업 카피본에 복수의 리비전 번호를 공존시키는 것을 들 수 있습니다.

처음에는 어쩌서 이러한 유연성이 장해가 아니라 기능이라고 생각되는 것인지 이해할 수 없을지도 모릅니다. 저장소(repository)로의 커밋이 완료되면 방금 커밋된 파일과 디렉토리가 나머지의 작업 카피본의 리비전보다 새로워집니다. 이것은 조금 혼란스럽게 보입니다. 전에 말한 것처럼 작업 카피본은 **svn update** 를 실행하는 것으로 항상 하나의 작업 리비전으로 할 수가 있습니다. 왜 **일부러** 다른 작업 리비전을 공존시키려고 할까요?

프로젝트가 매우 복잡한 경우를 생각해 보세요. 여러분은 작업 카피본의 일부를 예전 버전으로 강제로 "되돌리는" 일이 좋은 해결책인 경우가 많음을 알게될 것입니다. 어떻게 되돌리는지는 3장에서 설명합니다. 아마 여러분은 서버 디렉토리에 있는 서버 모듈의 이전의 버전을 테스트해보고 싶을 것입니다. 또는 최신의 버전하에서 하나의 파일의 이전의 여러가지 버전이 잘 움직이는지 확인하고 싶을 것입니다. "

작업 카피본 안에서 다른 리비전을 섞어서 이용할 수 있습니다만 유연성에 제한이 있습니다.

우선, 완전하게 최신이 아닌 파일이나 디렉토리의 삭제를 커밋 할 수 없습니다. 만약 새로운 버전의 파일이나 디렉토리가 저장소(repository)에 있다면 삭제하려는 시도는 거부됩니다. 이것은 여러분이 아직 알지 못한 변경 사항을 실수로 지워 버리는 것을 막으려는 것입니다.

그리고 최신으로 갱신하지 않으면 디렉토리의 메타데이터의 변경을 커

못 할 수 없습니다. 파일이나 디렉토리에 "속성"을 붙이는 방법에 대해서는 6장에서 설명합니다. 디렉토리의 작업 리비전은 특정한 엔트리와 속성의 집합을 정의하므로 최신이 아닌 디렉토리에 대한 속성 변경의 커밋은 아직 알아보지 못한 속성을 지워 버릴지도 모르기 때문입니다.

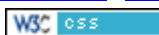
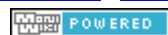
1.4. 개요

이 장에서는 Subversion의 기본적인 개념을 다루었습니다:

- 중심이 되는 저장소(repository) 클라이언트의 작업 카피본 저장소(repository) 리비전 트리라고 하는 개념을 도입했습니다.
- 어떻게 두 명의 공동 작업자가 subversion을 이용해 서로의 수정점을 공개하거나 받거나 할까의 간단한 예를 봐 왔습니다. 이것에는 복사·수정·머지 의 모델을 이용하므로 했다.
- Subversion이 작업 카피본내의 정보를 더듬거나 관리하거나 하는 방법에 대해 조금 접했습니다.

여기에서는 가장 일반적인 의미로 Subversion이 어떻게 동작할까에 대한 좋은 생각이 몸에 닿았을 것입니다. 이 지식을 기초로 다음의 장에 스스로 일이 할 수 있습니다. 여기는 Subversion의 커멘드와 기능에 대한 자세한 투어에 되어 있습니다.

[EditText](#) | [Refresh](#) | [FindPage](#) | [DeletePage](#) | [LikePages](#) | [Tour](#) | [Keywords](#) |

Last modified: 2005-06-01 13:33:20, Processing time: 0.0179 sec

[인터넷 익스플로러 7, 무료](#)

Google에 최적화. 향상된 보안 기능. 오늘 다운로드 받으세요!

[Subversion Made Better](#)

Transform subversion into a multi site development solution